

4×H200 NVL GPU Passthrough

OpenStack Nova — Deployment & Validation Report

Host	GPUs	Driver	Status
bgo-compute-h200-111	4 × NVIDIA H200 NVL (10de:233b)	595.71.05 / CUDA 13.2	VALIDATED

Summary

A single compute host carrying four NVIDIA H200 NVL GPUs was configured for full GPU passthrough under OpenStack Nova, with the goal of presenting all four cards to a single instance with working inter-GPU NVLink. The deployment is now complete and validated end to end: the four GPUs pass through cleanly, schedule onto the correct NUMA node, expose their full memory, and — critically — retain a full-mesh NVLink fabric inside the guest, confirmed by peer-to-peer bandwidth and latency testing.

NVLink fabric is fully healthy.

All four GPUs report 18 links each (Link 0–17), every one at 26.562 GB/s, none down or inactive — the exact H200 NVL signature of 18 NVLink lanes per GPU at ~26.5 GB/s each, aggregating to roughly 900 GB/s of bidirectional NVLink bandwidth per card. NVLink is up and running at full width and full speed on every card, with no degraded or dropped links. Combined with the NV6 full-mesh from topo -m, this confirms the passthrough preserved the complete NVLink topology. This is what a correctly working 4×H200 NVL rig looks like — and it was achieved without fabric manager, which is correct for direct-bridged NVL hardware.

Validated configuration

Layer	Result	Key detail
PCI passthrough	Working	Devices enumerate as type-PF; controller-side [pci] alias set to type-PF was the load-bearing fix
Scheduler	Placing	Aggregate membership + PciPassthroughFilter on controller; n1 flavor places GPU-local via required affinity
NUMA topology	Confirmed	All 4 GPUs affined to host NUMA node 1 (genuine HW affinity, sysfs-verified, not a BIOS artifact)
GPU memory	Full	143,771 MiB (~144 GB) visible per card, ECC enabled
NVLink	Full mesh	18 links/GPU @ 26.562 GB/s, NV6 mesh across all pairs
Fabric manager	Not required	Direct NVLink bridge (no NVSwitch); service correctly reports nothing to manage
Peer-to-peer	Verified	~131 GB/s unidirectional / ~260 GB/s bidirectional per pair with P2P enabled; ~2.7µs latency
Collective bandwidth	Verified	NCCL all_reduce ~322 GB/s busbw across 4 GPUs (peak); identical on both flavors, zero validation errors

Evidence

GPU inventory — nvidia-smi

Four H200 NVL devices present, each with full ~144 GB and ECC enabled, idle and healthy.

```

NVIDIA-SMI 595.71.05      Driver Version: 595.71.05      CUDA Version: 13.2
+-----+-----+-----+-----+
| GPU | Name           | Bus-Id           | Memory / ECC |
+-----+-----+-----+-----+
| 0   | NVIDIA H200 NVL | 00000000:06:00.0 | 0 / 143771 MiB ECC0 |
| 1   | NVIDIA H200 NVL | 00000000:07:00.0 | 0 / 143771 MiB ECC0 |
| 2   | NVIDIA H200 NVL | 00000000:08:00.0 | 0 / 143771 MiB ECC0 |
| 3   | NVIDIA H200 NVL | 00000000:09:00.0 | 0 / 143771 MiB ECC0 |
+-----+-----+-----+-----+
39C  P0  ~75W / 600W  0% util  No running processes found

```

Topology — `nvidia-smi topo -m`

Every GPU pair shows NV6 (6 bonded NVLinks) — a full mesh with no PCIe fallback (no SYS / PHB / PXB between GPUs). CPU/NUMA affinity 0–27, node 0 inside the guest (mapping to host node 1 where the cards physically sit).

```

      GPU0   GPU1   GPU2   GPU3   CPU Affinity   NUMA Affinity
GPU0      X     NV6    NV6    NV6      0-27           0
GPU1     NV6      X     NV6    NV6      0-27           0
GPU2     NV6     NV6      X     NV6      0-27           0
GPU3     NV6     NV6     NV6      X      0-27           0

NV# = bonded set of # NVLinks      (no SYS/PHB/PXB = no PCIe fallback)

```

Link status — `nvidia-smi nvlink --status`

Each GPU reports 18 active links (Link 0–17), all at 26.562 GB/s, none inactive.

```

GPU 0: NVIDIA H200 NVL   Link 0-17 : 26.562 GB/s   (all 18 links up)
GPU 1: NVIDIA H200 NVL   Link 0-17 : 26.562 GB/s   (all 18 links up)
GPU 2: NVIDIA H200 NVL   Link 0-17 : 26.562 GB/s   (all 18 links up)
GPU 3: NVIDIA H200 NVL   Link 0-17 : 26.562 GB/s   (all 18 links up)

18 links x 26.562 GB/s  ~=  900 GB/s bidirectional NVLink bandwidth per GPU

```

Peer-to-peer — `p2pBandwidthLatencyTest`

P2P access is enabled between all GPU pairs. Enabling P2P raises pairwise bandwidth from the ~37 GB/s PCIe path to ~131 GB/s (unidirectional) and ~260 GB/s (bidirectional), and drops latency from ~15–21µs to ~2.7µs — the unmistakable NVLink signature, uniform across every pair with no outliers. (Large diagonal values are each GPU's own HBM bandwidth.)

```

P2P Connectivity Matrix          all pairs = 1  (peer access enabled)

Unidirectional  P2P=Disabled (GB/s)      Unidirectional  P2P=Enabled (GB/s)
   0      1      2      3                0      1      2      3
0  3760  35.4  38.7  37.9                0  3795 131.3 130.9 130.9
1  34.9  3829  37.3  39.4                1  131.2 3821 131.0 130.9
2  37.9  37.9  3794  35.6                2  131.2 130.8 3799 130.9
3  37.9  37.7  35.5  3826                3  131.2 130.9 131.1 3833

Bidirectional   P2P=Enabled (GB/s)      Latency  P2P=Enabled (us)
   0      1      2      3                0      1      2      3
0  3922  260.8 260.4 260.5                0   1.91  2.68  2.69  2.68
1  260.8 3938  260.5 260.9                1   2.15  1.90  2.69  2.66
2  260.8 260.1 3920  260.7                2   2.67  2.17  2.01  2.69
3  260.9 260.5 260.0 3923                3   2.66  2.66  2.17  1.91

```

Note: the ~131 GB/s figure is below the link's theoretical ceiling because this sample uses a simple copy pattern not tuned for peak throughput — its purpose is to confirm P2P works over NVLink, which it definitively does. For peak aggregate bandwidth, `nccl-tests all_reduce_perf` (busbw) is the workload-representative measure.

Deployment path — what made it work

The build surfaced a sequence of distinct gates, each resolved in turn. The notes below double as a runbook for the next host.

1. PCI alias not defined on controller (HTTP 400)

`pci_passthrough:alias` is parsed by nova-api on the controller. The `[pci]` alias must exist there too, not only on the compute host.

2. Aggregate filter returned 0 hosts

Scheduler ran on a stale host→aggregate view; restarting nova-scheduler reloaded membership and the host became a candidate.

3. PciPassthroughFilter dropped 4 devices — mismatched attribute

Cards enumerate as `type-PF` (SR-IOV-capable), but the alias requested `type-PCI`. SR-IOV-capable devices must use `type-PF` even for whole-device passthrough. Fixed on both controller and compute, services restarted.

4. NUMATopologyFilter returned 0

All 4 GPUs sit on host NUMA node 1. Resolved by aligning the flavor's NUMA model and PCI affinity policy to the hardware (n1 = single-node/local; n2 = two-node/preferred).

5. Spawn failed — tap interface already exists

Leftover network state from a failed build + scheduler retries. Cleaned the orphaned tap and failed instance, then re-launched.

6. fabric manager service failed

Correct behaviour: no NVSwitch present on direct-bridged NVL hardware. Service disabled; NVLink was already up without it.

Flavors

Two flavors expose the locality-vs-capacity tradeoff and let the user choose. Both are private to the NREC project and request all four GPUs via the `h200nv1` alias.

Property	gr1.h200.nv1.n1	gr1.h200.nv1.n2
Intent	GPU-local, low latency, half the box	Max RAM/CPU, GPUs remote for half the vCPUs
vCPUs	28 (floating)	56 (floating)
RAM	368,640 MiB (360 GB)	737,280 MiB (720 GB)
hw:numa_nodes	1	2
pci_numa_affinity_policy	required	preferred
disk	1500 GB	1500 GB
GPUs	h200nv1:4	h200nv1:4

The two flavors are mutually exclusive on this single host: each claims all 4 GPUs, 360+ GB, and 1.5 TB of disk, so only one instance runs at a time.

Flavor comparison — n1 vs n2 (NCCL collectives)

Both flavors were benchmarked with NVIDIA nccl-tests (NCCL 2.30.7, CUDA 13.2) using identical parameters. The peak bus bandwidth (busbw) at the 8 GB message size — the figure that predicts multi-GPU

training scaling — is identical across the two flavors within run-to-run noise, with zero validation errors on every run. Because these collectives ride NVLink, which is GPU-to-GPU and independent of the guest CPU/NUMA layout, the flavor choice has no effect on multi-GPU performance.

Collective (4 GPU, peak busbw)	n1	n2	Verdict
all_reduce	~322.0 GB/s	~322.2 GB/s	identical
all_gather	~312.6 GB/s	~313.2 GB/s	identical
reduce_scatter	~307.3 GB/s	~307.4 GB/s	identical
all_reduce, 2 GPU	~118.3 GB/s	~118.4 GB/s	identical
all_reduce, 1 GPU*	~2076 GB/s	~2076 GB/s	identical

*1-GPU all_reduce reflects HBM3e memory bandwidth, not interconnect (busbw is undefined for a single rank). ~2 TB/s is healthy for H200. Note: the per-run "Avg bus bandwidth" line averages across all message sizes including tiny latency-bound ones, so it reads far lower (~116 GB/s) and is not the performance figure — the large-message plateau is.

The flavors differ structurally, but not in GPU performance.

virsh confirms the placement difference: **n1** pins all 28 vCPUs to host NUMA node 1 (numa_nodeset: 1, cores 16-31,48-63) — the node where all four GPUs physically sit, so every vCPU is GPU-local. **n2** spans both nodes (numa_nodeset: 0-1), giving 56 vCPUs and 720 GB, but half its vCPUs are one NUMA hop from the GPUs for host-side transfers. The NCCL results above show this placement difference does not affect multi-GPU collective bandwidth.

Guidance for users

Choose between n1 and n2 on CPU/RAM needs, not GPU performance — multi-GPU NVLink throughput is identical. Pick **n1** when the workload fits in 360 GB and full per-vCPU GPU locality is wanted; pick **n2** when more RAM/CPU is needed (large datasets, heavy CPU preprocessing, large host-side buffers) and the minor cross-node cost on host transfers is acceptable.

Productization checklist

- **Golden image:** install the NVIDIA datacenter driver (595.71.05) and the CUDA toolkit at the version your users' frameworks expect (13.2 works driver-wise; 12.x may suit prebuilt ML wheels better).
- **Disable fabric manager in the image** so instances do not boot a failing service unit. It is not needed on direct-bridged NVL hardware.
- **Put CUDA on PATH** via /etc/profile.d/cuda.sh so every user gets nvcc and the runtime libraries without manual setup.
- **Lock in the flavor pair** (n1 / n2) with NREC project-access grants.
- **Document the topology dependency:** fabric manager is not required for NVL-bridged GPUs, but IS required for any future NVSwitch / HGX H200 nodes.